



Лекція 14

Введення у Node.js

Express

Node.js — це кросплатформне середовище виконання JavaScript. Працює на рушії V8 JavaScript (ядро Google Chrome), поза браузером, призначений для створення серверних програм на мові JavaScript.

Node.js використовує один (основний) потік, який отримує всі запити та керує ними через чергу запитів (таким чином, Node.js є однопотоковим сервером). Усередині цього потоку виконується так званий цикл подій (event loop), який безперервно перевіряє запити з черги подій і обробляє події введення та виведення.

При виконанні операцій введення-виведення, таких як читання з мережі, доступ до бази даних або файлової системи, замість блокування потоку та витрачання циклів ЦП на очікування, Node.js відновить операції, коли повернеться відповідь.

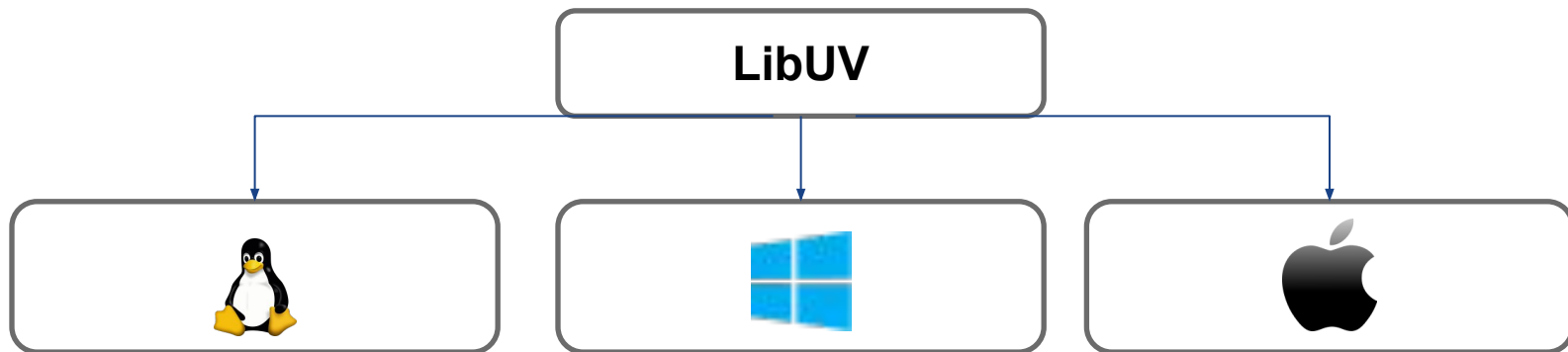
Це дозволяє Node.js обробляти тисячі одночасних з'єднань з одним сервером, не вносячи тягаря керування паралельністю потоків, що може бути значним джерелом помилок.

<https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>

Переваги використання

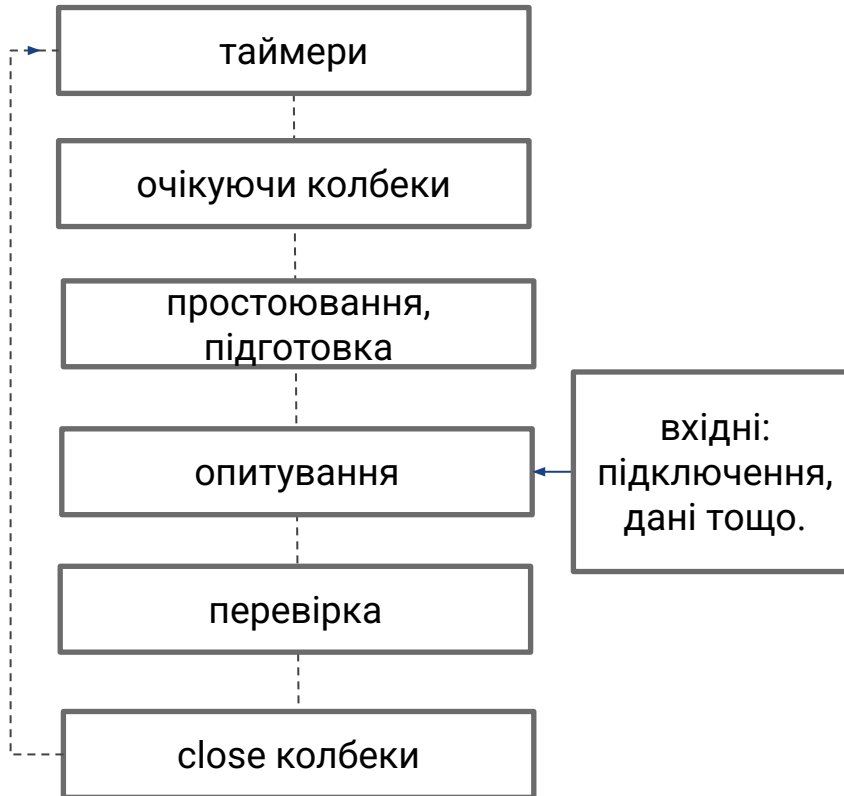
- 1. Швидкодія:** Node.js використовує асинхронний, неблокуючий ввід/вивід, що дозволяє ефективно обробляти велику кількість одночасних підключень без блокування виконання інших запитів. Це дозволяє створювати високоефективні серверні додатки.
- 2. Єдина мова:** дозволяє розробникам використовувати ту ж мову програмування і на серверній, і на клієнтській стороні, що спрощує розробку та підтримку коду.
- 3. Велика спільнота та екосистема:** Node.js має велику та активну спільноту розробників. Це означає, що є безліч корисних модулів (через npm - менеджер пакетів Node.js), фреймворків та інструментів для розробки різноманітних додатків.
- 4. Мінімально життєздатний продукт (Minimum Viable Product):** Основна ідея MVP - швидко випустити продукт на ринок, перевірити його прийняття користувачами і вчасно здійснювати необхідні корективи.
- 5. Масштабованість:** Асинхронна природа Node.js дозволяє ефективно використовувати ресурси системи та масштабувати додатки для обробки великого обсягу запитів. Завдяки модульності Node.js розвивається швидко, дозволяючи розробникам легко розширювати функціональність за допомогою сторонніх пакетів та власних модулів.

Кросплатформеність



- відповідає за управління потоками I/O.
- містить Event Loop що дозволяє Node.js виконувати операції асинхронно.

Event Loop



- ця фаза виконує заплановані зворотні виклики, `setTimeout()` та `setInterval()`.

- виконує зворотні виклики I/O, відкладені до наступної ітерації циклу.

- використовувати лише внутрішньо.

- отримання нових подій I/O; виконувати зворотні виклики, пов'язані з введенням/виведенням (майже всі, за винятком зворотних викликів закриття, запланованих таймерами та `setImmediate()`); вузол блокуватиме тут, коли це буде потрібно.

- `setImmediate()` тут викликаються зворотні виклики.

- close callbacks : деякі закриті зворотні виклики, наприклад `socket.on('close', ...)`

Перший додаток

Створити файл `app.js` у вибраному каталозі та додати код

```
const http = require('http');

http.createServer((request, response) => {
  response.end('Good morning NodeJS!');
}).listen(3000, () => {
  console.log('Started working on port 3000');
});
```

Відкрити термінал у каталозі зі створеним файлом, виконати:
`node app.js`

package.json

`npm init` - створює файл `package.json`.

Додавши `--yes` або скоротшення `-y` призведе до автоматичного створення файлу `package.json` з усіма значеннями за замовчуванням без будь-яких питань або підтверджень.

```
{
  "name": "nodejs-intro-01-firts-app",
  "version": "1.0.0",
  "description": "",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Модулі

Модулі в Node.js є основним будівельним блоком при розробці додатків. Вони дозволяють організувати код у логічні, перевикористовувані блоки, що полегшує розробку, підтримку та масштабування програмного забезпечення. Node.js надає можливість імпортувати модулі з вбудованими або зовнішніми бібліотек, а також створювати власні модулі за допомогою системи модульного вводу/виводу CommonJS. Цей слайд розгляне ключові аспекти використання модулів в Node.js та їх важливість для побудови структурованих та ефективних додатків.

CommonJS		ES Modules
Synchronous	Module Loading	Asynchronous
'require'/'exports'	Import/Export	'import'/'export'
Cached	Caching	Not cached by default
Server-side	Use Case	Client-side and server-side
Not supported	Dynamic Import	Supported

CommonJS

user.js

```
const userData = {  
  name: "John",  
  age: 23  
};  
  
function greetUser() {  
  console.log(`Hi, ${userData.name}!`);  
}  
  
module.exports = {  
  userData,  
  greetUser,  
};
```

service.js

```
const {  
  userData,  
  greetUser,  
} = require('./user.js');  
  
console.log(userData);  
greetUser();
```

ES Modules

user.js

```
const userData = {  
  name: "John",  
  age: 23  
};  
  
function greetUser() {  
  console.log(`Hi, ${userData.name}!`);  
}  
  
export { userData, greetUser };
```

service.mjs

```
import {  
  userData,  
  greetUser,  
} from './user.mjs';  
  
console.log(userData);  
greetUser();
```

Об'єкт global та глобальні змінні

Ці об'єкти доступні у всіх модулях.

У Node.js є кілька вбудованих глобальних об'єктів, які можуть бути корисні для розробки. Ось декілька з них:

process: Цей об'єкт надає інформацію про поточний процес Node.js, таку як аргументи командного рядка, змінні середовища та інше.

console: Об'єкт console використовується для виводу повідомлень у консоль. Він має методи, такі як log, error, warn та інші.

Buffer: Цей об'єкт дозволяє робити операції з буферами (наприклад, робота з бінарними даними).

setTimeout і **setInterval**: Ці функції дозволяють встановлювати таймери для виконання коду через певний інтервал часу.

лише в модулях CommonJS:

```
{__dirname, __filename, exports, module, require() };
```

<https://nodejs.org/api/globals.html>

Консольні додатки

Консольні додатки в Node.js - це програми, які виконуються в терміналі або командному рядку. Вони дозволяють взаємодіяти з користувачем або автоматизувати завдання через текстовий інтерфейс.

Передача параметрів додатку

Для отримання параметрів у коді програми застосовується масив `process.argv`. Це аналогічно до того, як Java у функцію `main` передається набір аргументів як рядкового масиву.

Перший елемент рядку вказує на шлях виконуючий програми `node.exe`. Другий елемент масиву завжди вказує на шлях до файлу програми, який виконується.

```
let nodePath = process.argv[0];
let appPath = process.argv[1];
let name = process.argv[2];
let age = process.argv[3];

console.log('nodePath: ' + nodePath);    // nodePath: C:\Program Files\nodejs\node.exe
console.log('appPath: ' + appPath);      // appPath: D:\profitsoft\index.js
console.log();                          //
console.log('name: ' + name);            // name: John
console.log('age: ' + age);              // age: 23
```

```
node index.js John 23
```

Консольні додатки

readline - вбудований модуль в Node.js дозволяє зчитувати введення користувача з терміналу. Використовується для створення інтерактивних консольних програм.

- **createInterface(options)**: Створює новий інтерфейс readline. options - об'єкт параметрів, де можна вказати властивості input та output для встановлення потоку вводу та виводу відповідно.
- **question(query, callback)**: Показує query користувачу, чекає на введення від нього та викликає callback з введеним текстом.
- **close()**: Закриває інтерфейс readline, звільняє ресурси.
- **on(event, callback)**: Додає обробник події event з функцією callback.
- **pause()**: Призупиняє читання вводу
- **resume()**: Відновити читання вводу

```
const rl = require('readline').createInterface({
  input: process.stdin,
  output: process.stdout
});

rl.question('Як вас звати? ', (name) => {
  console.log(`Привіт, ${name}!`);
  rl.pause();
});

// інші дії, поки чекаємо на відповідь користувача
rl.resume();
```

Консольні додатки

commander: бібліотека надає можливість легко створювати консольні інтерфейси для ваших програм, включаючи опції командного рядка та підтримку аргументів.

```
const { Command } = require('commander');

const program = new Command();
program
  .version('1.0.0')
  .description('Командна програма на Node.js');

program
  .command('reverse <string>')
  .description('Реверс рядка')
  .action((str) => {
    const reversedStr = str.split('').reverse().join('');
    console.log(`Реверсований рядок: ${reversedStr}`);
  });

program
  .command('toUpperCase <string>')
  .description('Перетворити рядок в верхній регістр')
  .action((str) => {
    const upperCaseStr = str.toUpperCase();
    console.log(`Рядок у верхньому регістрі: ${upperCaseStr}`);
  });

program.parse(process.argv);
```

```
node index.js reverse "Nice"
node index.js toUpperCase "Nice"
```

```
// Реверсований рядок: eciN
// Рядок у верхньому регістрі: NICE
```

Nodemon

Що таке Nodemon?

- Nodemon - це інструмент для розробки Node.js, який допомагає автоматизувати процес перезапуску сервера під час змін в файловій системі.

Як він працює?

- Моніторить зміни в файлах вашого проекту та автоматично перезапускає сервер при виявленні змін.

Основні переваги:

- Економія часу розробника.
- Автоматичний процес перезапуску.
- Підтримка живого оновлення під час розробки.

Як встановити?

- Додайте Nodemon як залежність: `npm install --save-dev nodemon`

Як використовувати?

- Додати script у вашому package.json на "dev": "nodemon app.js", а потім запустіть сервер командою `npm run dev`.

Робота з файлами

Node.js надає потужні інструменти для роботи з файловою системою.

Вбудований модуль `fs` (file system) дозволяє виконувати різні операції з файлами та каталогами.

Деякі з основних операцій, які можна виконувати за допомогою модуля `fs`:

- Читання файлів (`fs.readFile`, `fs.readFileSync`)
- Запис у файли (`fs.writeFile`, `fs.writeFileSync`)
- Видалення файлів (`fs.unlink`, `fs.unlinkSync`)
- Створення каталогів (`fs.mkdir`, `fs.mkdirSync`)
- Видалення каталогів (`fs.rmdir`, `fs.rmdirSync`)
- Перевірка існування файлів та каталогів (`fs.existsSync`)

```
const fs = require('fs');

fs.writeFile('hello.txt', 'Hello! Be wise!', (err) => {
  if (err) console.error('Помилка запису:', err);
  fs.readFile('hello.txt', 'utf8', (err, data) => {
    if (err) {
      console.error('Помилка читання файлу:', err);
      return;
    }
    console.log('Вміст файлу: ', data);
  });
});
```

Node.js побудований на основі подій, що дозволяє ефективно реагувати на асинхронні події.

EventEmitter - ключовий модуль для генерації та прослуховування подій.

on() - використовується для реєстрації обробника подій.

once() - реєструє обробник, який буде викликаний лише один раз для певної події.

emit() - викликає всі зареєстровані обробники для певної події.

Причому однієї події можна вказати безліч обробників. Коли об'єкт **EventEmitter** генерує подію, відбувається виконання всіх обробників

Передача параметрів події

При виклику події як другий параметр у функцію **emit** можна передавати певний об'єкт, який передається у функцію обробника події.

Успадкування від EventEmitter

У додатку ми можемо оперувати складними об'єктами, для яких можна визначати події, але для цього їх треба пов'язати з об'єктом **EventEmitter**.


```
const EventEmitter = require('events');
const footballChampionship = new EventEmitter();

footballChampionship.once('championshipStart', () => console.log('Чемпіонат розпочався!'));

// Подія: Початок матчу (відбувається кожного разу перед початком матчу)
footballChampionship.on('matchStart', (team1, team2) => {
  console.log(`Початок матчу: ${team1} проти ${team2}`);
});

// Подія: Гол (відбувається кожного разу, коли забивають гол)
footballChampionship.on('goal', (team, player) => {
  console.log(`Гол! Гравець ${player} забив гол за команду ${team}`);
});

// Подія: Закінчення матчу (відбувається кожного разу після завершення матчу)
footballChampionship.on('matchEnd', (team1, team2, winner) => {
  console.log(`Матч закінчився! Переможець: ${winner}`);
});

// Подія: Закінчення чемпіонату (відбувається тільки один раз)
footballChampionship.once('championshipEnd', () => console.log('Чемпіонат завершився!'));

// Старт чемпіонату
footballChampionship.emit('championshipStart');

// Старт матчів
footballChampionship.emit('matchStart', 'Збірна А', 'Збірна Б');
footballChampionship.emit('goal', 'Збірна А', 'Даниленко');
footballChampionship.emit('goal', 'Збірна Б', 'Олійник');
footballChampionship.emit('goal', 'Збірна А', 'Кравченко');
footballChampionship.emit('matchEnd', 'Збірна А', 'Збірна Б', 'Збірна А');

footballChampionship.emit('matchStart', 'Збірна В', 'Збірна Г');
footballChampionship.emit('goal', 'Збірна В', 'Коваленко');
footballChampionship.emit('goal', 'Збірна Г', 'Ткаченко');
footballChampionship.emit('goal', 'Збірна В', 'Шевченко');
footballChampionship.emit('matchEnd', 'Збірна В', 'Збірна Г', 'Збірна В');

footballChampionship.emit('championshipEnd');
```

Stream

Stream - це інтерфейс в Node.js для роботи з потоками даних, який дозволяє читати або записувати дані партіями, а не в одному блоку.

Типи потоків

1. **Readable** (читання): Потік, з якого можна читати дані.
2. **Writable** (запис): Потік, в який можна записувати дані.
3. **Duplex** (дуплекс): Потік, який може працювати як зчитування, так і запис.
4. **Transform** (перетворення): Потік, для трансформації даних під час читання та запису.

Переваги використання Stream

1. **Ефективність**: Потоки дозволяють оптимізувати використання пам'яті та швидкість обробки даних.
2. **Масштабованість**: За допомогою потоків можна обробляти великі обсяги даних без перевищення обсягу доступної пам'яті.
3. **Постійна доступність**: Потоки дозволяють обробляти дані по мірі їх надходження, що сприяє постійній доступності системи.

Використання Stream в Node.js дозволяє покращити продуктивність та ефективність обробки даних, особливо коли маємо справу з великими обсягами інформації.

Stream

```
const fs = require('fs');

// Створення потоку для читання файлу
const readableStream = fs.createReadStream('input.txt');

// Створення потоку для запису у файл
const writableStream = fs.createWriteStream('output.txt');

// Підключення обробників подій до потоків
readableStream.on('data', (chunk) => {
  console.log(chunk);
  // Обробка отриманих даних
  writableStream.write(chunk); // Запис даних у файл
});

readableStream.on('end', () => {
  console.log('Читання завершено');
  writableStream.end(); // Завершення запису у файл
});
```

Pipe - це канал, який пов'язує потік для читання та потік для запису і дозволяє відразу зчитувати з потоку читання в потік запису. .

Основні особливості:

- **Потокова обробка даних**: Дані можуть бути передані в потоки і оброблені безпосередньо в процесі їх отримання, що зменшує необхідність у тимчасовому зберіганні у пам'яті.
- **Ефективність ресурсів**: Потоки дозволяють оптимізувати ресурси, так як дані обробляються порціями, а не всі одночасно.
- **Модульність і гнучкість**: Потоки можуть бути з'єднані в будь-якому порядку, що дозволяє створювати складні конвеєри обробки даних.

Pipe

```
const fs = require('fs');
const zlib = require('zlib');

// Створюємо потік для читання вхідного файлу
const readStream = fs.createReadStream('input.txt');

// Створюємо потік для запису стиснутого файлу
const writeStream = fs.createWriteStream('input.txt.gz');

// Створюємо архіватор
const gzip = zlib.createGzip();

// Підключаємо потоки до архіватора
readStream.pipe(gzip).pipe(writeStream);

// Обробляємо подію завершення архівування
writeStream.on('close', () => {
  console.log('Архівування завершено!');
});
```

Створення сервера

```
const http = require('http');

const PORT = process.env.PORT || 3000;
http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write(`
    <!DOCTYPE html>
    <html lang="en">
    <head>
      <meta charset="UTF-8">
      <meta name="viewport" content="width=device-width, initial-scale=1.0">
      <title>Simple Node.js</title>
      <style>
        body {
          display: flex;
          justify-content: center;
        }
        .content { text-align: center }
      </style>
    </head>
    <body>
      <div class="content">
        <h1>Привіт з сервера Node.js!</h1>
        <p>Це HTML відповідь від нашого сервера.</p>
      </div>
    </body>
    </html>
  `);
  res.end();
}).listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Маршрутизація

```
const http = require('http');
const PORT = process.env.PORT || 3000;

http.createServer((req, res) => {
  if (req.url === '/') {
    res.write(`
      <h1>Home page</h1>
      <a href="/services"><button>Services</button></a>
      <a href="/team"><button>Our team</button></a>
      <a href="/about"><button>About us</button></a>
    `);
    res.end();
  } else if (req.url === '/services') {
    res.write(`
      <h1>Services</h1><a href="/"><button>Home page</button></a>
    `);
    res.end();
  } else if (req.url === '/team') {
    res.write(`
      <h1>Our team</h1><a href="/"><button>Home page</button></a>
    `);
    res.end();
  } else if (req.url === '/about') {
    res.write(`
      <h1>About</h1><a href="/"><button>Home page</button></a>
    `);
    res.end();
  } else {
    res.write('<h1>404 Not Found</h1>');
    res.end();
  }
}).listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Надсилання статичних файлів

При використанні `fs.createReadStream()`, дані передаються клієнту асинхронно через потік, що забезпечує більш ефективне використання ресурсів пам'яті та покращену швидкість передачі даних.

```
const http = require('http');
const fs = require('fs');
const path = require('path');

const PORT = process.env.PORT || 3000;

http.createServer((request, response) => {
  const filePath = request.url.substr(1);
  if (filePath === '') {
    const filePath = path.join(__dirname, 'index.html');
    const stream = fs.createReadStream(filePath);
    stream.pipe(response);
    stream.on('error', (error) => {
      console.error('Error reading file:', error);
    });
  } else {
    response.writeHead(404, {'Content-Type': 'text/html'});
    response.write('<h1>404 Not Found</h1>');
    response.end();
  }
})
.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```


Надсилання статичних файлів

При використанні `fs.readFile()`, яке є асинхронною операцією, файл повністю зчитується у пам'ять перед тим, як дані будуть відправлені клієнту. Це може призвести до проблем з продуктивністю при роботі з великими файлами або при великій кількості запитів.

```
const http = require('http');
const fs = require('fs');
const path = require('path');

const PORT = process.env.PORT || 3000;

http.createServer((request, response) => {
  const filePath = request.url.substr(1);
  if (filePath === '') {
    const filePath = path.join(__dirname, 'index.html');
    fs.readFile(filePath, (error, data) => {
      if (error) {
        console.error('Error reading file:', error);
      } else {
        response.writeHead(200, {'Content-Type': 'text/html'});
        response.end(data);
      }
    });
  } else {
    response.writeHead(404, {'Content-Type': 'text/html'});
    response.write('<h1>404 Not Found</h1>');
    response.end();
  }
})
.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Шаблони

Шаблони - це візуальні або текстові елементи, які використовуються для створення консистентного вигляду сторінок.

Вони дозволяють розділити логіку додатку та представлення даних, що спрощує розробку та підтримку коду.

Переваги використання шаблонів:

- ✓ **Перевикористання коду:** Шаблони дозволяють використовувати одну й ту ж HTML-структуру для різних сторінок.
- ✓ **Покращена підтримка:** Зберігання роздільності між логікою та представленням полегшує зміни та розширення.
- ✓ **Легша розробка:** Шаблони дозволяють розробникам швидше створювати нові сторінки або компоненти, оскільки вони можуть використовувати існуючі шаблони.

Популярні бібліотеки шаблонів для Node.js:

Handlebars: Простий у використанні та потужний шаблонізатор, який дозволяє використовувати шаблони для HTML, CSS, JS і багато іншого.

EJS (Embedded JavaScript): Дозволяє вбудовувати JavaScript код безпосередньо в HTML, що робить його дуже гнучким.

Express

`Express.js` - це веб-фреймворк для Node.js, що дозволяє швидко та легко створювати веб-додатки та API.

Він надає простий та елегантний спосіб обробки запитів та відправлення відповідей.

```
npm install express
```

```
const express = require('express');
const app = express();

// Маршрут для відправлення текстового повідомлення на кореневий URL
app.get('/', (req, res) => {
  res.send('Привіт, світ!');
});

// Маршрут для відправлення HTML сторінки
app.get('/about', (req, res) => {
  res.send('<h1>Про нас</h1><p>Це сторінка про наш веб-сайт.</p>');
});

const port = 3000;
app.listen(port, () => console.log(`Сервер запущено на порті ${port}`));
```

Методи екземпляра `express` (об'єкт додатка):

`get(path, callback)`: Визначає обробник для GET-запитів за певним маршрутом.

`post(path, callback)`: Визначає обробник для POST-запитів за певним маршрутом.

`put(path, callback)`: Визначає обробник для PUT-запитів за певним маршрутом.

`patch(path, callback)`: Визначає обробник для PATCH-запитів за певним маршрутом.

`delete(path, callback)`: Визначає обробник для DELETE-запитів за певним маршрутом.

`all(path, callback)`: Визначає обробник для будь-якого типу запиту (GET, POST, PUT, DELETE) за певним маршрутом.

`use([path,] callback)`: Встановлює проміжний обробник для всіх типів запитів або для певного маршруту.

`listen(port, [hostname], [backlog], [callback])`: Прослуховує певний порт та запускає сервер.

`set(setting, value)`: Встановлює налаштування додатка.

Middleware

Функції проміжної обробки (**middleware**) - це функції, що мають доступ до об'єкта запиту (req), об'єкта відповіді (res) та до наступної функції проміжної обробки в циклі 'запит-відповідь' програми. Наступна функція проміжної обробки, як правило, позначається змінною next.

Функції проміжної обробки можуть виконувати такі завдання:

- ✓ Виконання будь-якого коду.
- ✓ Внесення змін до об'єктів запитів та відповідей.
- ✓ Завершення циклу 'запит-відповідь'.
- ✓ Виклик наступного проміжного оброблювача зі стека.
- ✓ Якщо поточна функція проміжної обробки не завершує цикл запиту-відповідь, вона повинна викликати next() для передачі керування наступній функції проміжної обробки.

```
const loggingMiddleware = (req, res, next) => {  
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);  
  next();  
};  
app.use(loggingMiddleware);  
app.get('/hello', (req, res) => {  
  res.send('Привіт, світ!');  
});  
app.listen(port, () => console.log(`Сервер запущено на порті ${port}`));
```

Express. Router.

Маршрутизатор (Router) у Express - це об'єкт, який допомагає управляти маршрутами веб-запитів у вашому додатку. Він дозволяє групувати обробники маршрутів для певних шляхів, щоб зробити ваш код більш організованим і легким для розуміння.

```
// app.js
const express = require('express');
const cardsRouter = require('./routers/api/cards');

const app = express();
app.use('/api/cards', cardsRouter);

app.listen(3000, () => console.log(`Server running`));

// path: /routes/api/cards
const express = require('express');

const router = express.Router();

router.get('/', async (req, res) => {
  const result = await cardService.getAllCards();
  res.json(result);
})

module.exports = router;
```

Express. Методи об'єкта запиту (req)

req.params: Об'єкт, що містить параметри шляху.

req.query: Об'єкт, що містить параметри URL-запиту.

req.body: Об'єкт, що містить дані запиту (тіло запиту), які зазвичай використовуються для POST або PUT запитів.

req.headers: Об'єкт, що містить заголовки HTTP-запиту.

req.method: Рядок, що містить метод HTTP-запиту (GET, POST, PUT, DELETE тощо).

req.url: Рядок, що містить повний URL-адресу запиту, включаючи шлях і параметри.

req.path: Рядок, що містить шлях URL-запиту.

req.cookies: Об'єкт, що містить cookies, що надійшли з клієнта.

req.protocol: Рядок, що містить протокол HTTP або HTTPS.

Express. Методи об'єкта відповіді (res)

`res.send(data)`: Відправляє відповідь клієнту з даними.

`res.json(data)`: Відправляє JSON-відповідь клієнту.

`res.status(code)`: Встановлює статус HTTP-відповіді.

`res.redirect(url)`: Перенаправляє клієнта на іншу URL-адресу.

`res.render(view, data)`: Рендерить HTML-шаблон за допомогою шаблонізатора.

`res.cookie(name, value, options)`: Встановлює cookie на стороні клієнта.

`res.clearCookie(name)`: Видаляє cookie з клієнта.

`res.header(header, value)`: Встановлює заголовок відповіді.

Express. Обробка помилок

Функції проміжного обробника обробки помилок визначаються як і інші функції проміжної обробки, але із зазначенням функції обробки помилок не трьох, а чотирьох аргументів: (err, req, res, next).

Проміжний обробник для обробки помилок повинен бути визначений останнім, після вказівки всіх `app.use()` та викликів маршрутів

```
app.use(function(err, req, res, next) {  
  console.error(err.stack);  
  res.status(500).send('Something broke!');  
});  
  
app.listen(3000, () => console.log(`Server running`));
```

CORS

CORS (Cross-Origin Resource Sharing) - це механізм безпеки веб-браузера, який дозволяє серверам контролювати, як веб-додатки на інших доменах можуть отримувати доступ до їх ресурсів. У Node.js ми можемо використовувати певні пакети або налаштування, щоб управляти політикою CORS на нашому сервері.

1. Проблема CORS: Без належної конфігурації сервера, запити між різними доменами можуть бути заборонені браузером з міркувань безпеки.

2. Рішення CORS в Node.js: Встановлення правильних заголовків CORS на сервері Node.js для дозволу запитів від інших доменів.

```
const express = require('express');
const cors = require('cors');

const app = express();

// Використання middleware CORS
app.use(cors());

// Запуск сервера
const port = 3000;
app.listen(port, () => {
  console.log(`Сервер запущено на порті ${port}`);
});
```



Дякуємо за увагу!